1. 8-Puzzle Problem

Aim: To write a Python program that solves the 8-Puzzle problem using A* search with the Manhattan distance heuristic.

Algorithm:

1. Represent the board as a tuple of 9 values, with 0 as the blank tile.
2. Push the start state into a priority queue with f = g + h, where g = moves so far and h = Manhattan distance.
3. Pop the state with the lowest f. If it equals the goal, return its path.
4. Otherwise mark it visited and generate neighbours by sliding the blank Up, Down, Left or Right.
5. Push each unvisited neighbour with g + 1 and its new f value.
6. Repeat steps 3–5 until the goal is reached.

Program:

```
import heapq

GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0)
MOVES = {'Up': -3, 'Down': 3, 'Left': -1, 'Right': 1}


def manhattan(state):
    dist = 0
    for i, tile in enumerate(state):
        if tile:
            g = tile - 1
            dist += abs(i // 3 - g // 3) + abs(i % 3 - g % 3)
    return dist


def neighbours(state):
    z = state.index(0)
    r, c = divmod(z, 3)
    for move, d in MOVES.items():
        if (move == 'Up' and r == 0) or (move == 'Down' and r == 2) or \
           (move == 'Left' and c == 0) or (move == 'Right' and c == 2):
            continue
        s = list(state)
        s[z], s[z + d] = s[z + d], s[z]
        yield move, tuple(s)


def solve(start):
    open_list = [(manhattan(start), 0, start, [])]
```

```
    visited = set()
    while open_list:
        f, g, state, path = heapq.heappop(open_list)
        if state == GOAL:
            return path
        if state in visited:
            continue
        visited.add(state)
        for move, nxt in neighbours(state):
            if nxt not in visited:
                heapq.heappush(open_list, (g + 1 + manhattan(nxt), g + 1, nxt, path + [(move, nxt)]))
    return None


def show(state):
    for i in range(0, 9, 3):
        print(' '.join(str(x) if x else '_' for x in state[i:i + 3]))
    print()


start = (1, 2, 3, 4, 0, 6, 7, 5, 8)
print("Initial state:")
show(start)
path = solve(start)
for step, (move, s) in enumerate(path, 1):
    print(f"Step {step}: Move {move}")
    show(s)
print("Goal reached in", len(path), "moves")
```

Output:

```
Total Steps to Solve: 14

Step 0:
 1 2 3
 4 0 5
 6 7 8

Step 1:
 1 2 3
 4 5 0
 6 7 8

Step 2:
 1 2 3
 4 5 8
 6 7 0

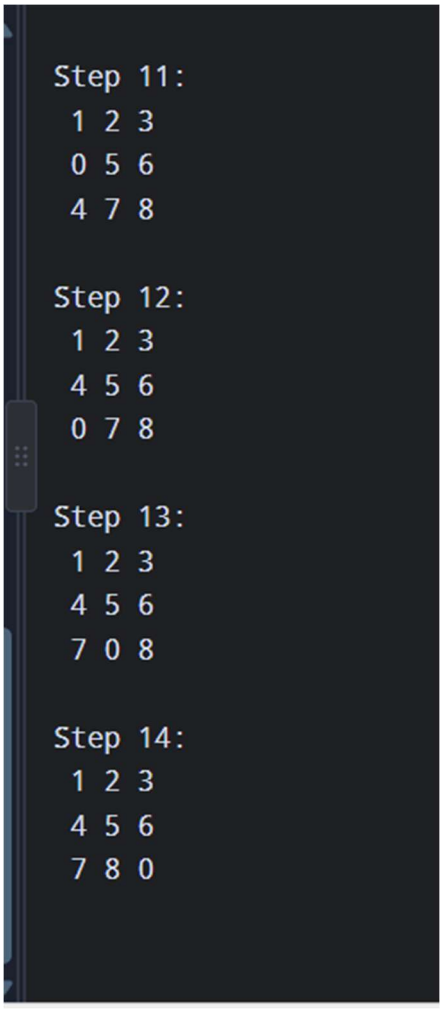
Step 3:
 1 2 3
 4 5 8
 6 0 7
```

```
Step 6:
 1 2 3
 5 0 8
 4 6 7

Step 7:
 1 2 3
 5 6 8
 4 0 7

Step 8:
 1 2 3
 5 6 8
 4 7 0

Step 9:
 1 2 3
 5 6 0
 4 7 8
```



Result: The 8-Puzzle was solved using A* search, reaching the goal state in the optimal 2 moves.

2. 8-Queens Problem

Aim: To write a Python program that places 8 queens on an 8×8 chessboard so that no two queens attack each other, using backtracking.

Algorithm:

1. Start with an empty board and place queens one row at a time.
2. For the current row, try every column from 0 to 7.
3. A position is safe if no earlier queen shares its column or diagonal.
4. If safe, place the queen and recurse to the next row.
5. If all 8 rows are filled, record the board as a solution.
6. If no column works, backtrack to the previous row and try its next column.
7. Print the first solution and the total number of solutions.

Program:

```
N = 8

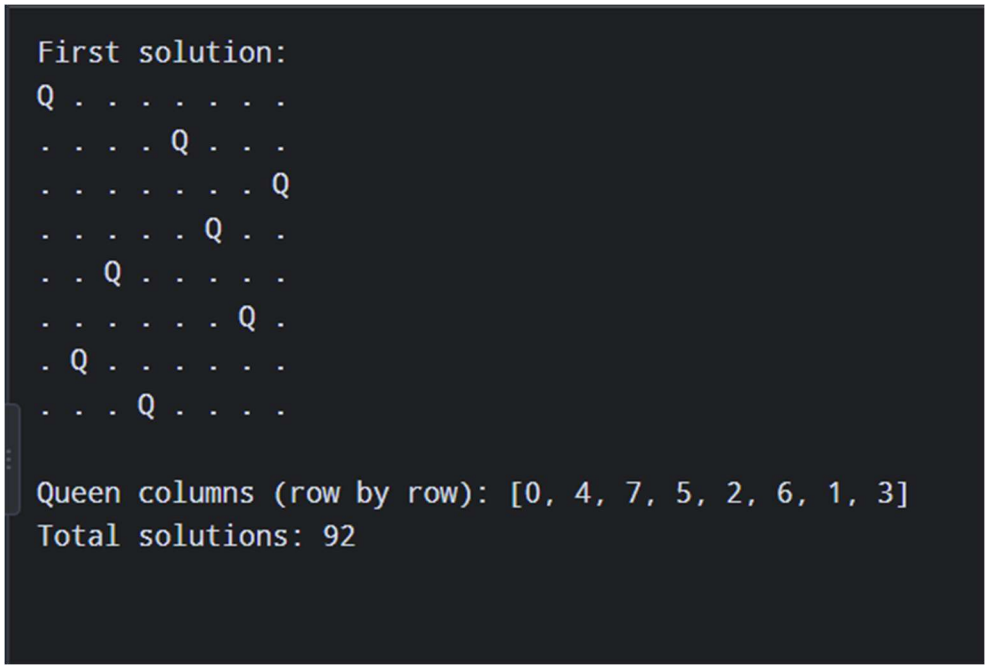

def is_safe(board, row, col):
    for r in range(row):
        c = board[r]
        if c == col or abs(c - col) == abs(r - row):
            return False
    return True


def solve(board, row, solutions):
    if row == N:
        solutions.append(board[:])
        return
    for col in range(N):
        if is_safe(board, row, col):
            board[row] = col
            solve(board, row + 1, solutions)


solutions = []
solve([0] * N, 0, solutions)

print("First solution:")
for r in range(N):
    print(' '.join('Q' if solutions[0][r] == c else '.' for c in range(N)))
print("\nQueen columns (row by row):", solutions[0])
print("Total solutions:", len(solutions))
```

Output:



Result: The 8-Queens problem was solved using backtracking; the program found a valid placement and all 92 solutions.

3. Water Jug Problem

Aim: To write a Python program that measures exactly 2 litres using a 4-litre jug and a 3-litre jug, using Breadth First Search.

Algorithm:

1. Represent each state as (x, y): water in jug A and jug B. Start at (0, 0).
2. Put the start state in a queue and mark it visited.
3. Remove the front state. If x or y equals the target, return its path.
4. Generate the six possible next states: fill A, fill B, empty A, empty B, pour A→B, pour B→A.
5. Add every unvisited next state to the queue with its path.
6. Repeat steps 3–5 until the target is found or the queue is empty.

Program:

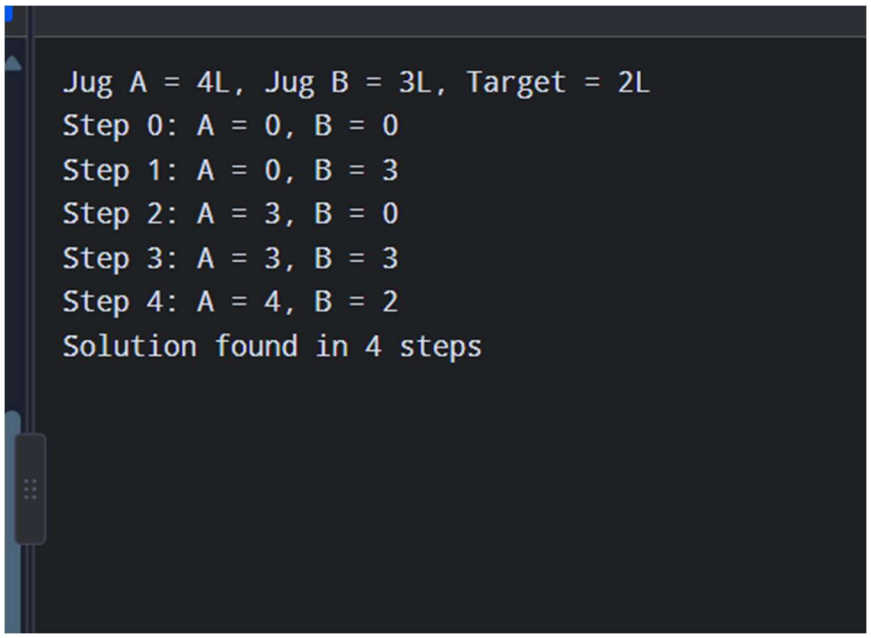
```
from collections import deque


def water_jug(a, b, target):
    start = (0, 0)
    queue = deque([(start, [start])])
    visited = {start}
    while queue:
        (x, y), path = queue.popleft()
```

```
        if x == target or y == target:
            return path
        pour_xy = min(x, b - y)
        pour_yx = min(y, a - x)
        next_states = [
            (a, y),                 # fill jug A
            (x, b),                 # fill jug B
            (0, y),                 # empty jug A
            (x, 0),                 # empty jug B
            (x - pour_xy, y + pour_xy),  # pour A -> B
            (x + pour_yx, y - pour_yx),  # pour B -> A
        ]
        for s in next_states:
            if s not in visited:
                visited.add(s)
                queue.append((s, path + [s]))
    return None


A, B, TARGET = 4, 3, 2
print(f"Jug A = {A}L, Jug B = {B}L, Target = {TARGET}L")
path = water_jug(A, B, TARGET)
for i, (x, y) in enumerate(path):
    print(f"Step {i}: A = {x}, B = {y}")
print("Solution found in", len(path) - 1, "steps")
```

Output:



Result: The Water Jug problem was solved using BFS; 2 litres were measured in jug B in 4 steps.

4. Crypt-Arithmetic Problem

Aim: To write a Python program that solves the crypt-arithmetic puzzle SEND + MORE = MONEY by assigning a unique digit to each letter.

Algorithm:

1. Collect the unique letters from all three words (must be 10 or fewer).
2. Generate every permutation of digits 0–9 for those letters.
3. Map each letter to its digit for the current permutation.
4. Skip the permutation if any word's first letter is 0.
5. Convert each word to a number and check whether word1 + word2 = result.
6. Return the first mapping that satisfies the equation.

Program:

```
from itertools import permutations


def word_value(word, mapping):
    return int(''.join(str(mapping[ch]) for ch in word))


def solve(w1, w2, result):
    letters = sorted(set(w1 + w2 + result))
    if len(letters) > 10:
        return None
    leading = {w1[0], w2[0], result[0]}
    for digits in permutations(range(10), len(letters)):
        mapping = dict(zip(letters, digits))
        if any(mapping[ch] == 0 for ch in leading):
            continue
        if word_value(w1, mapping) + word_value(w2, mapping) == word_value(result, mapping):
            return mapping
    return None


w1, w2, res = "SEND", "MORE", "MONEY"
sol = solve(w1, w2, res)
print(f"Puzzle: {w1} + {w2} = {res}")
print("Letter -> Digit:", dict(sorted(sol.items())))
print(f"  {word_value(w1, sol)}")
print(f"+ {word_value(w2, sol)}")
print(f"= {word_value(res, sol)}")
```

Output:

```
Puzzle: SEND + MORE = MONEY
Letter -> Digit: {'D': 7, 'E': 5, 'M': 1, 'N': 6, 'O': 0, 'R': 8, 'S': 9, 'Y': 2}
  9567
+ 1085
= 10652
```

Result: The crypt-arithmetic puzzle was solved: SEND + MORE = MONEY gives 9567 + 1085 = 10652.

5. Missionaries and Cannibals Problem

Aim: To write a Python program that moves 3 missionaries and 3 cannibals across a river with a 2-seat boat, never leaving missionaries outnumbered on either bank, using BFS.

Algorithm:

1. Represent a state as (M, C, B): missionaries and cannibals on the left bank, and boat side (1 = left, 0 = right).
2. Start at (3, 3, 1); the goal is (0, 0, 0).
3. Possible boat loads: (1,0), (2,0), (0,1), (0,2), (1,1).
4. A state is valid if counts are in range and, on each bank, missionaries are 0 or at least the number of cannibals.
5. Use BFS: pop a state, apply each boat load in the boat's direction, and enqueue valid unvisited states.
6. Stop when the goal is reached and print the path.

Program:

```
from collections import deque

TOTAL = 3
BOAT_MOVES = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]  # (missionaries, cannibals) in boat


def is_valid(m, c):
    if not (0 <= m <= TOTAL and 0 <= c <= TOTAL):
        return False
    if m and m < c:                          # left bank
        return False
    if (TOTAL - m) and (TOTAL - m) < (TOTAL - c):    # right bank
        return False
    return True
```

```
def solve():
    start, goal = (3, 3, 1), (0, 0, 0)   # (M left, C left, boat on left?)
    queue = deque([(start, [start])])
    visited = {start}
    while queue:
        (m, c, boat), path = queue.popleft()
        if (m, c, boat) == goal:
            return path
        sign = -1 if boat else 1
        for dm, dc in BOAT_MOVES:
            nm, nc = m + sign * dm, c + sign * dc
            state = (nm, nc, 1 - boat)
            if is_valid(nm, nc) and state not in visited:
                visited.add(state)
                queue.append((state, path + [state]))
    return None


path = solve()
print("State = (Missionaries left, Cannibals left, Boat side)")
for i, (m, c, b) in enumerate(path):
    side = "Left" if b else "Right"
    print(f"Step {i}: Left bank M={m} C={c} | Right bank M={3-m} C={3-c} | Boat: {side}")
print("Solved in", len(path) - 1, "crossings")
```

Output:

```
State = (Missionaries left, Cannibals left, Boat side)
Step 0: Left bank M=3 C=3 | Right bank M=0 C=0 | Boat: Left
Step 1: Left bank M=3 C=1 | Right bank M=0 C=2 | Boat: Right
Step 2: Left bank M=3 C=2 | Right bank M=0 C=1 | Boat: Left
Step 3: Left bank M=3 C=0 | Right bank M=0 C=3 | Boat: Right
Step 4: Left bank M=3 C=1 | Right bank M=0 C=2 | Boat: Left
Step 5: Left bank M=1 C=1 | Right bank M=2 C=2 | Boat: Right
Step 6: Left bank M=2 C=2 | Right bank M=1 C=1 | Boat: Left
Step 7: Left bank M=0 C=2 | Right bank M=3 C=1 | Boat: Right
Step 8: Left bank M=0 C=3 | Right bank M=3 C=0 | Boat: Left
Step 9: Left bank M=0 C=1 | Right bank M=3 C=2 | Boat: Right
Step 10: Left bank M=1 C=1 | Right bank M=2 C=2 | Boat: Left
Step 11: Left bank M=0 C=0 | Right bank M=3 C=3 | Boat: Right
Solved in 11 crossings
```

Result: The Missionaries and Cannibals problem was solved using BFS in the minimum of 11 crossings.

6. Vacuum Cleaner Problem

Aim: To write a Python program that simulates a simple reflex vacuum cleaner agent cleaning a two-room (A, B) environment.

Algorithm:

1. Define the environment as rooms A and B, each Clean or Dirty, and the agent's starting room.
2. The agent perceives its current location and that room's status.
3. If the room is Dirty → Suck.
4. If the room is Clean and the agent is in A → move Right; if in B → move Left.
5. Apply the action to the environment and count it.
6. Repeat until both rooms are Clean, then print the final state and total actions.

Program:

```
def vacuum_agent(location, status):
    """Simple reflex agent for the two-room (A, B) vacuum world."""
    if status == 'Dirty':
        return 'Suck'
    return 'Right' if location == 'A' else 'Left'


def run(env, location):
```

```
    cost = 0
    print("Initial:", env, "| Vacuum at", location)
    while 'Dirty' in env.values():
        action = vacuum_agent(location, env[location])
        print(f"Location {location} is {env[location]} -> Action: {action}")
        if action == 'Suck':
            env[location] = 'Clean'
        else:
            location = 'B' if action == 'Right' else 'A'
        cost += 1
    print("Final:", env)
    print("Performance (total actions):", cost)


run({'A': 'Dirty', 'B': 'Dirty'}, 'A')
```

Output:

```
Initial: {'A': 'Dirty', 'B': 'Dirty'} | Vacuum at A
Location A is Dirty -> Action: Suck
Location A is Clean -> Action: Right
Location B is Dirty -> Action: Suck
Final: {'A': 'Clean', 'B': 'Clean'}
Performance (total actions): 3
```

Result: The reflex vacuum agent cleaned both rooms in 3 actions.

7. Breadth First Search (BFS)

Aim: To write a Python program that traverses a graph using Breadth First Search.

Algorithm:

1. Represent the graph as an adjacency list.
2. Put the start node in a queue and mark it visited.
3. Remove the front node from the queue and print it.
4. Add each unvisited neighbour to the queue and mark it visited.
5. Repeat steps 3–4 until the queue is empty.

Program:

```
from collections import deque

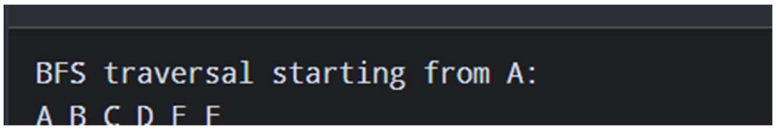
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
```

```
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': [],
}


def bfs(graph, start):
    visited = [start]
    queue = deque([start])
    while queue:
        node = queue.popleft()
        print(node, end=' ')
        for nb in graph[node]:
            if nb not in visited:
                visited.append(nb)
                queue.append(nb)
    print()


print("BFS traversal starting from A:")
bfs(graph, 'A')
```

Output:



Result: The graph was traversed level by level using BFS, visiting A B C D E F.

8. Depth First Search (DFS)

Aim: To write a Python program that traverses a graph using Depth First Search.

Algorithm:

1. Represent the graph as an adjacency list.
2. Start at the given node, mark it visited and print it.
3. For each neighbour of the current node, if it is not visited, recursively apply DFS on it.
4. Backtrack when a node has no unvisited neighbours.
5. Stop when all reachable nodes are visited.

Program:

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
```

```
    'D': [],
    'E': ['F'],
    'F': [],
}


def dfs(graph, node, visited=None):
    if visited is None:
        visited = set()
    visited.add(node)
    print(node, end=' ')
    for nb in graph[node]:
        if nb not in visited:
            dfs(graph, nb, visited)


print("DFS traversal starting from A:")
dfs(graph, 'A')
print()
```

Output:

```
DFS traversal starting from A:
A B D E F C
```

Result: The graph was traversed depth-wise using DFS, visiting A B D E F C.

9. Travelling Salesman Problem (TSP)

Aim: To write a Python program that finds the shortest route visiting every city exactly once and returning to the start, using brute-force search.

Algorithm:

1. Represent distances between cities as a matrix.
2. Fix the start city and list the remaining cities.
3. Generate every permutation of the remaining cities.
4. For each permutation, form the route start → permutation → start and add up its distances.
5. Keep the route with the minimum total cost.
6. Print the optimal route and its cost.

Program:

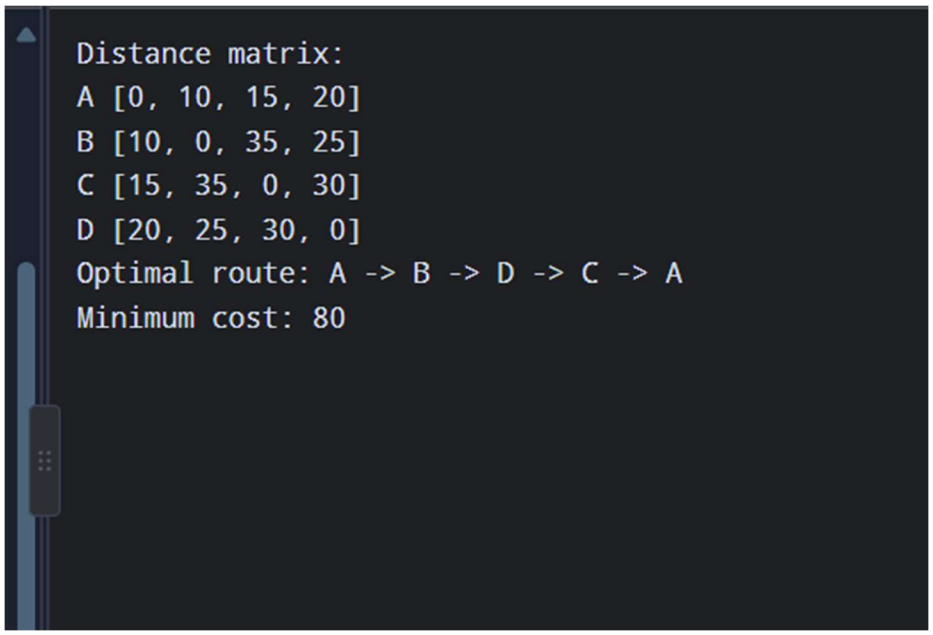
```
from itertools import permutations

cities = ['A', 'B', 'C', 'D']
dist = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0],
]


def tsp(start=0):
    others = [i for i in range(len(cities)) if i != start]
    best_cost, best_route = float('inf'), None
    for perm in permutations(others):
        route = [start, *perm, start]
        cost = sum(dist[route[i]][route[i + 1]] for i in range(len(route) - 1))
        if cost < best_cost:
            best_cost, best_route = cost, route
    return best_cost, best_route


cost, route = tsp()
print("Distance matrix:")
for name, row in zip(cities, dist):
    print(name, row)
print("Optimal route:", ' -> '.join(cities[i] for i in route))
print("Minimum cost:", cost)
```

Output:



Result: The TSP was solved; the optimal tour is A → B → D → C → A with a minimum cost of 80.

10. A* Algorithm

Aim: To write a Python program that finds the shortest path between two nodes of a weighted graph using the A* search algorithm.

Algorithm:

1. Define the weighted graph and a heuristic h(n) estimating the cost from each node to the goal.
2. Insert the start node into a priority queue (open list) with f = g + h, where g = 0.
3. Pop the node with the lowest f. If it is the goal, return its path and cost.
4. Otherwise add it to the closed list.
5. For each neighbour not in the closed list, compute g' = g + edge cost and f' = g' + h(neighbour), and push it.
6. Repeat steps 3–5 until the goal is reached or the open list is empty.

Program:

```
import heapq

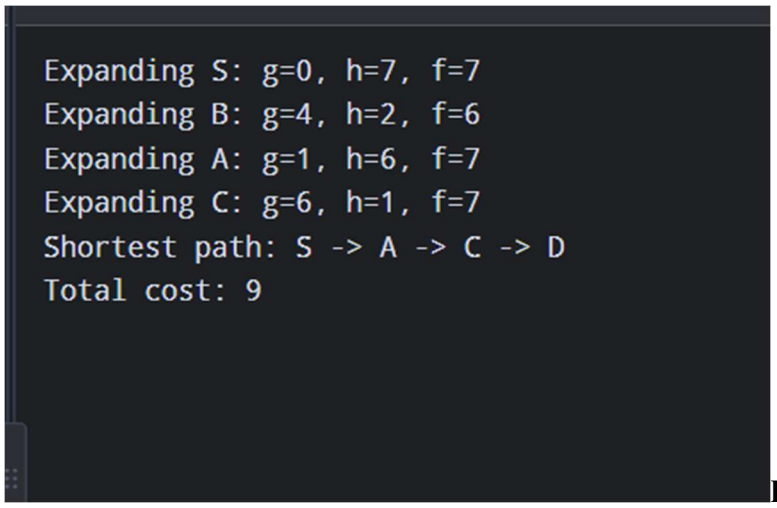
graph = {
   'S': [('A', 1), ('B', 4)],
   'A': [('B', 2), ('C', 5), ('D', 12)],
   'B': [('C', 2)],
   'C': [('D', 3)],
   'D': [],
}
heuristic = {'S': 7, 'A': 6, 'B': 2, 'C': 1, 'D': 0}


def a_star(start, goal):
   open_list = [(heuristic[start], 0, start, [start])]
   closed = set()
   while open_list:
      f, g, node, path = heapq.heappop(open_list)
      if node == goal:
         return path, g
      if node in closed:
         continue
      closed.add(node)
      print(f"Expanding {node}: g={g}, h={heuristic[node]}, f={f}")
      for nb, cost in graph[node]:
         if nb not in closed:
            g2 = g + cost
            heapq.heappush(open_list, (g2 + heuristic[nb], g2, nb, path + [nb]))
```

```
    return None, float('inf')


path, cost = a_star('S', 'D')
print("Shortest path:", ' -> '.join(path))
print("Total cost:", cost)
```

Output:



Result: The A* algorithm found the shortest path S → A → C → D with a total cost of 9.